

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente "CUNOC"
División Ciencias de la Ingeniería
Laboratorio de Lenguajes Formales y de Programación
Ing. Daniel Gonzalez



Manual Técnico PromptZal - Analizador Léxico

William Eduardo Gómez Sánchez
Carné: 202232064

Guatemala, Quetzaltenango, 17 de agosto de 2026

Introducción

El presente documento describe los aspectos técnicos del analizador léxico desarrollado para el lenguaje PromptZal, como parte de la práctica del Laboratorio de Lenguajes Formales y de Programación.

PromptZal es un lenguaje diseñado para que los programadores puedan redactar prompts estructurados de forma rápida, los cuales son utilizados posteriormente en herramientas de inteligencia artificial.

El analizador léxico desarrollado constituye la primera fase de un compilador, encargada de leer un archivo con extensión .pz y dividirlo en componentes mínimos con significado (tokens), clasificándolos según su categoría.

El programa fue desarrollado en Java, recorriendo el texto de entrada carácter por carácter mediante lógica propia, e identificando cada categoría de token a través de reglas definidas manualmente.

Requisitos del Sistema

Requisitos mínimos para ejecutar el proyecto:

- Sistema Operativo: Windows, Linux o macOS
- Java Development Kit (JDK): Versión 17 o superior
- Memoria RAM: 512 MB mínimo
- Espacio en disco: 50 MB mínimo
- Un archivo de texto con extensión .pz para analizar

Para compilar el proyecto:

- Apache Maven 3.6 o superior
- NetBeans IDE 17 o superior (recomendado)

Para ejecutar el JAR:

```
java -jar Promptzal-1.0-SNAPSHOT.jar
```

Para compilar desde NetBeans:

1. Abrir el proyecto en NetBeans
2. Click derecho en el proyecto
3. Seleccionar "Clean and Build"
4. El JAR se genera en la carpeta target

Para compilar desde consola:

1. Abrir CMD o PowerShell
2. Navegar a la carpeta raíz del proyecto: `cd ruta\del\proyecto`
3. Ejecutar: `mvn clean package`
4. El JAR se genera en la carpeta target

Para ejecutar desde consola:

1. Navegar a la carpeta target:
`cd ruta\del\proyecto\target`

2. Ejecutar:

```
java -jar Promptzal-1.0-SNAPSHOT.jar
```

Descripción de Tecnologías

Lenguaje de programación:

- **Java 17** — Lenguaje principal del proyecto, utilizado para implementar toda la lógica del analizador léxico.

Herramientas de desarrollo:

- **NetBeans IDE 17** — Entorno de desarrollo utilizado para escribir, compilar y depurar el código.
- **Apache Maven** — Herramienta de gestión y construcción del proyecto, utilizada para compilar y generar el archivo JAR ejecutable.
- **draw.io** — Herramienta utilizada para elaborar el diagrama de clases del proyecto.

Librería gráfica:

- **Java Swing** — Utilizada únicamente para desplegar el selector de archivos (JFileChooser), que permite al usuario elegir el archivo .pz a analizar de forma visual.

Formato de salida:

- **HTML** — Utilizado para generar los reportes de tokens y de errores léxicos, mostrados en tablas legibles dentro de un navegador web.

Características técnicas:

- **Manejo de archivos** — Se utiliza `Files.readAllBytes()` para leer el contenido completo del archivo .pz, y `FileWriter` para generar los reportes HTML.
- **Colecciones** — Se utiliza `List` (implementada con `ArrayList`) para almacenar los tokens y errores léxicos encontrados durante el análisis, ya que su cantidad no se conoce de antemano.

Conceptos de POO aplicados:

- **Encapsulamiento** — Los atributos de las clases `Token` y `ErrorLexico` son privados, con acceso mediante getters y setters.
- **Manejo de excepciones** — Se maneja `IOException` para el caso en que el archivo seleccionado no pueda ser leído correctamente.

Arquitectura del Proyecto

El proyecto está organizado en dos paquetes principales:

app: Contiene la clase principal `Promptzal` (con el método `main`), la clase `AnalizadorLexico` (encargada de todo el proceso de análisis léxico) y la clase `GeneradorReporte` (encargada de generar los reportes en formato HTML).

modelo: Contiene las clases de dominio Token y ErrorLexico, que representan respectivamente un componente léxico reconocido y un error encontrado durante el análisis.

vista: Contiene la clase Colores, utilizada para dar formato visual mediante códigos ANSI a la salida en consola.

Paquetes del proyecto:

- **app** → Clase principal, analizador léxico y generador de reportes.

- **modelo** → Clases de dominio: Token y ErrorLexico.

- **vista** → Clase de apoyo visual: Colores.

Descripción de Clases

Paquete **app**

- **Promptzal** — Clase principal del proyecto. Contiene el método main que muestra la bienvenida, abre el selector de archivos, y coordina el flujo completo: lee el archivo, ejecuta el análisis léxico, muestra los tokens en consola y genera los reportes HTML.

- **AnalizadorLexico** — Clase central del proyecto, encargada de recorrer el texto de entrada carácter por carácter y clasificar cada componente en su categoría correspondiente. Contiene los métodos de reconocimiento para palabras, directivas, cadenas, números, comentarios, el conector "->" y los símbolos sueltos, además de los métodos de apoyo espiar() y avanzar() para el recorrido del texto.

- **GeneradorReporte** — Clase encargada de generar los archivos HTML con la tabla de tokens reconocidos y la tabla de errores léxicos encontrados durante el análisis.

Paquete **modelo**

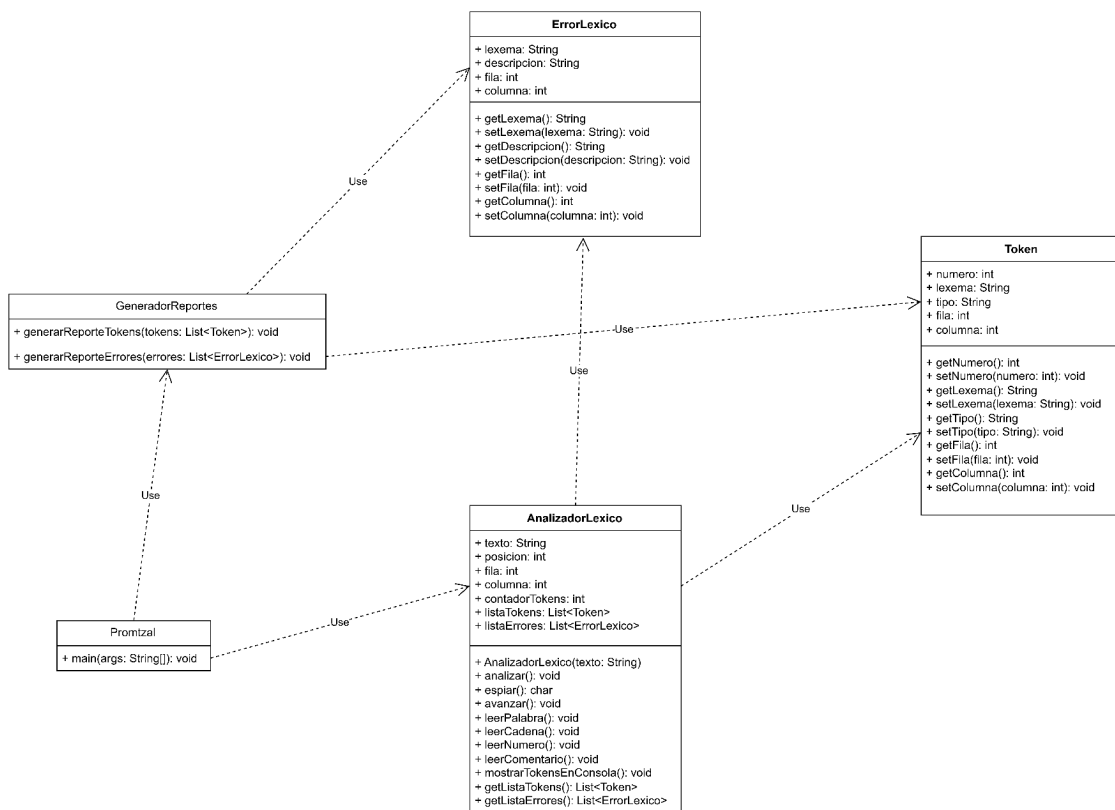
- **Token** — Clase que representa un componente léxico reconocido. Contiene el número correlativo, el lexema, el tipo, la fila y la columna donde fue encontrado.

- **ErrorLexico** — Clase que representa un error léxico encontrado durante el análisis. Contiene el lexema o carácter problemático, una descripción del error, la fila y la columna.

Paquete vista

- Colores — Clase que contiene las constantes de códigos de escape ANSI utilizadas para dar formato de color y estilo al texto mostrado en la consola.

Diagrama de Clases



Para visualizar el diagrama completo también se puede acceder al siguiente enlace:

<https://drive.google.com/file/d/1QoXpoViSqhsERFKxmMQ6SFdZEZYi6J2F/view?usp=sharing>

Lista de Tokens

El analizador léxico reconoce las siguientes categorías de tokens:

Categoría	Ejemplos	Tipo Asignado
Directivas	@modelo, @rol, @formato	DIRECTIVA
Palabras reservadas	AGENTE, contexto, variable, EJECUTAR, EXPORTAR	PALABRA_RESERVADA
Comandos de IA	PREGUNTAR, GENERAR, RESUMIR, ANALIZAR, TRADUCIR, CLASIFICAR, EXTRAER, CARGAR	COMANDO_IA
Conectores	SOBRE, DESDE, EN, COMO, ->	CONECTOR
Identificadores	analista, ventas, resultado	IDENTIFICADOR
Literales de cadena	"texto entre comillas"	CADENA
Literales enteros	10, 100	ENTERO
Literales decimales	0.7, 3.14	DECIMAL
Operadores	=, +	OPERADOR
Delimitadores	{ } () , "	DELIMITADOR
Comentarios	// línea, /* bloque */	(se ignoran, no generan token)

Descripción del Recorrido

El analizador léxico recorre el texto de entrada de principio a fin, carácter por carácter, mediante un ciclo principal implementado en el método `analizar()`. En cada posición, se examina el carácter actual y, según su naturaleza, se delega el reconocimiento al método correspondiente:

Si el carácter es una letra o guion bajo, se invoca el método `leerPalabra()`, que acumula caracteres mientras sean letras, dígitos o guiones bajos, y clasifica la palabra resultante como palabra reservada, comando de IA, conector o identificador, según listas predefinidas.

Si el carácter es un arroba (@), se invoca `leerDirectiva()`, que funciona de forma similar a `leerPalabra()` pero omitiendo el símbolo inicial.

Si el carácter es una comilla ("), se invoca `leerCadena()`, que acumula el contenido hasta encontrar la comilla de cierre. Si se alcanza un salto de línea o el final del archivo antes de encontrar el cierre, se reporta como error léxico.

Si el carácter es un dígito, se invoca `leerNumero()`, que acumula dígitos y, opcionalmente, un único punto decimal seguido de más dígitos, clasificando el resultado como entero o decimal.

Si el carácter es una diagonal (/), se invoca `leerComentario()`, que determina si se trata de un comentario de línea o de bloque, descartando su contenido sin generar ningún token. Si un comentario de bloque no encuentra su cierre, se reporta como error léxico.

Si el carácter es un guion (-), se examina el siguiente carácter mediante el método `espiar()`; si corresponde a un mayor que (>), se reconoce el conector "->". Los espacios en blanco y saltos de línea se omiten sin generar ningún token, actualizando únicamente la fila y columna actuales.

Cualquier carácter que no encaje en ninguna de las categorías anteriores se reporta como un error léxico de carácter no reconocido, sin detener el análisis del resto del archivo.

Manejo de Errores Léxicos

El analizador contempla dos tipos de error léxico, según lo especificado en el enunciado:

Carácter no reconocido: ocurre cuando el analizador encuentra un símbolo que no corresponde a ninguna categoría válida del lenguaje (por ejemplo, #, \$, %). En este caso, se registra un error con el carácter problemático, su fila y columna, y el análisis continúa avanzando a partir del siguiente carácter.

Cadena sin cerrar: ocurre cuando se encuentra una comilla de apertura (") pero el analizador llega a un salto de línea o al final del archivo sin haber encontrado la comilla de cierre correspondiente. Se registra un error con el contenido leído hasta el momento del fallo, la fila y la columna donde comenzó la cadena.

Adicionalmente, y como criterio propio de diseño, se contempla un tercer caso: un comentario de bloque (`/* */`) que no encuentra su cierre correspondiente también se reporta

como error léxico, ya que representa una inconsistencia similar en la sintaxis del archivo de entrada.

En todos los casos, el análisis nunca se detiene ante un error; el recorrido continúa hasta procesar la totalidad del archivo, acumulando todos los errores encontrados en una lista independiente de la lista de tokens reconocidos.

Pseudocódigo de métodos importantes

Método espiar()

espiar()

Si posicion + 1 es menor que la longitud del texto entonces
retornar el caracter en la posicion + 1

Si no

retornar caracter nulo

Fin espiar

Método avanzar()

avanzar()

Si el caracter en la posicion actual es salto de linea entonces
incrementar fila

columna = 1

Si no

incrementar columna

incrementar posicion

Fin avanzar

Método leerCadena()

leerCadena()

guardar fila y columna de inicio

avanzar() // se salta la comilla de apertura

cerrada = falso

Mientras haya texto por leer hacer

Si el caracter actual es comilla entonces

avanzar() // se salta la comilla de cierre

cerrada = verdadero

salir del ciclo

Si el caracter actual es salto de linea entonces

salir del ciclo

```
    acumular el caracter actual
    avanzar()
FinMientras
Si cerrada es verdadero entonces
    crear Token de tipo CADENA con lo acumulado
Si no
    crear ErrorLexico "Cadena sin cerrar"
Fin leerCadena
```

Método leerNumero()

```
leerNumero()
    guardar fila y columna de inicio
    esDecimal = falso
    Mientras haya texto por leer hacer
        Si el caracter actual es digito entonces
            acumular el caracter, avanzar()
        Si no, si el caracter actual es punto Y no esDecimal
            Y el siguiente caracter (espiado) es digito entonces
                esDecimal = verdadero
                acumular el punto, avanzar()
        Si no
            salir del ciclo
    FinMientras
    Si esDecimal entonces
        crear Token de tipo DECIMAL
    Si no
        crear Token de tipo ENTERO
    Fin leerNumero
```

Método analizar()

```
analizar()
    Mientras haya texto por leer hacer
        caracter_actual = obtener caracter en posicion actual
        Si es espacio o salto de linea entonces
            avanzar()
        Si no, si es letra o guion bajo entonces
            leerPalabra()
        Si no, si es arroba entonces
            leerDirectiva()
        Si no, si es comilla entonces
            leerCadena()
        Si no, si es digito entonces
            leerNumero()
```

```
Si no, si es diagonal entonces
    leerComentario()
Si no, si es guion entonces
    leerConectorFlecha()
Si no, si es simbolo suelto valido entonces
    leerSimboloSuelto()
Si no
    crear ErrorLexico "Caracter no reconocido"
    avanzar()
FinMientras
Fin analizar
```

Conclusión

El desarrollo del analizador léxico para PromptZal permitió aplicar de forma práctica los fundamentos de la primera fase de un compilador, implementando desde cero el reconocimiento de cada categoría de token mediante lógica propia.

El diseño del recorrido carácter por carácter, junto con la técnica de anticipación (lookahead) para casos como el conector "->" y la diferenciación entre comentarios de línea y de bloque, permitió construir un tokenizador robusto capaz de clasificar correctamente las nueve categorías de tokens definidas por el lenguaje.

La implementación del manejo de errores léxicos, capaz de continuar el análisis tras encontrar un error sin detener el procesamiento del resto del archivo, refuerza la utilidad práctica del programa como una herramienta que informa de manera completa sobre la validez de un archivo de entrada, en lugar de fallar ante el primer inconveniente encontrado.

Como resultado final se obtuvo un analizador léxico funcional, acompañado de reportes exportables a formato HTML tanto para los tokens reconocidos como para los errores detectados, cumpliendo con los requerimientos establecidos para esta primera entrega del proyecto.